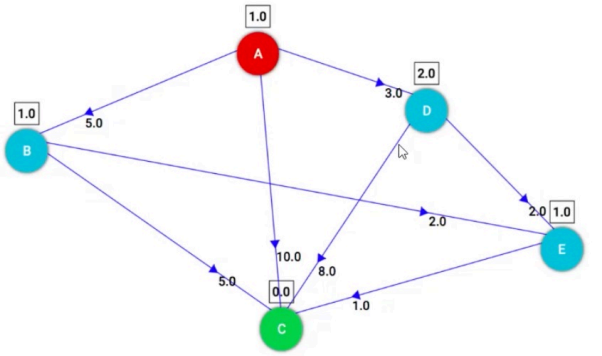
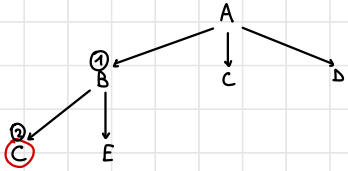
30 /09 /24

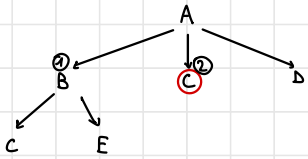


1) DEPTH - First SEARCH



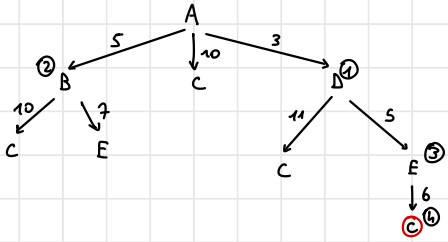
COST = 5 + 5 = 10

2) BREADTH - First SEARCH



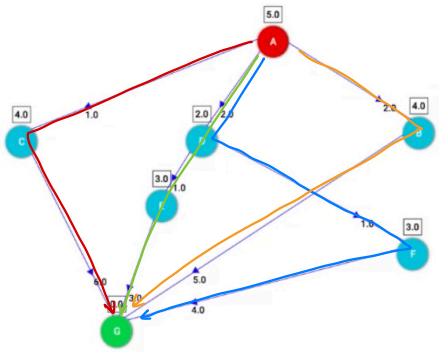
COST = 10

3) UNIFORM - COST SEARCH



COST = 6

15/10/24



1) Heuristic admissible ?

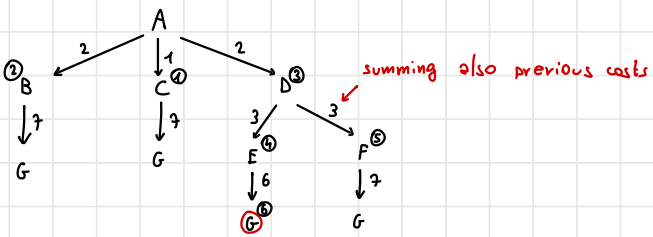
- $S \geq 1+6$
- $S \geq 2+1+3$
- $S \geq 2+1+1$
- $S \geq 2+5$

} Always optimistic ⟶ Yes

3) A*

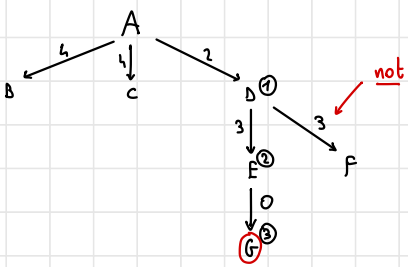
1) UNIFORM-COST SEARCH

only path cost



2) BEST-FIRST SEARCH / HILL CLIMBING

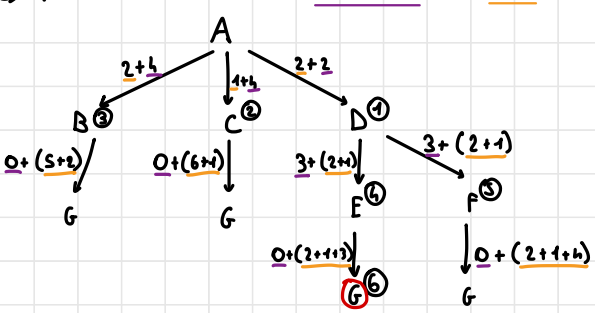
only heuristic



Δ By chance finds the optimal solution

3) A* ALGORITHM

heuristic + cost



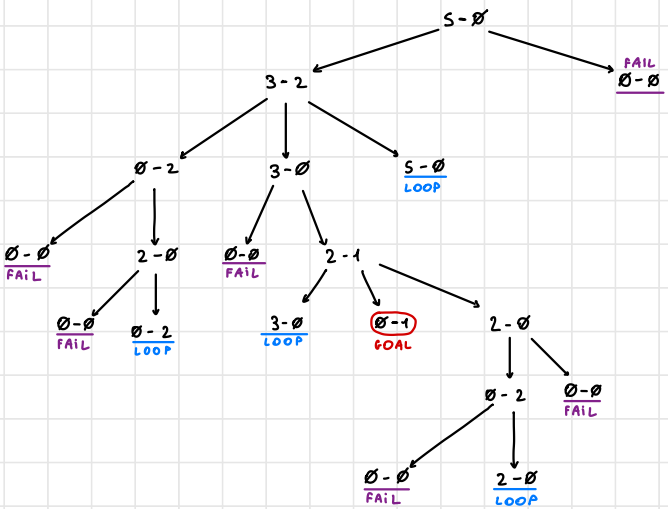
16/10/24

We have two bins: the first has capacity of 5 liters and is full of water, the second has capacity of 2 liters and is empty.

- We want to obtain one liter of water in the second bin.
- We can transfer water from one bin to the other, we can throw water away, but we cannot obtain new water.
- Show the search space of the problem. Explain after how many steps we reach the goal using breadth-first and depth-first.

INITIAL STATE : S - Ø

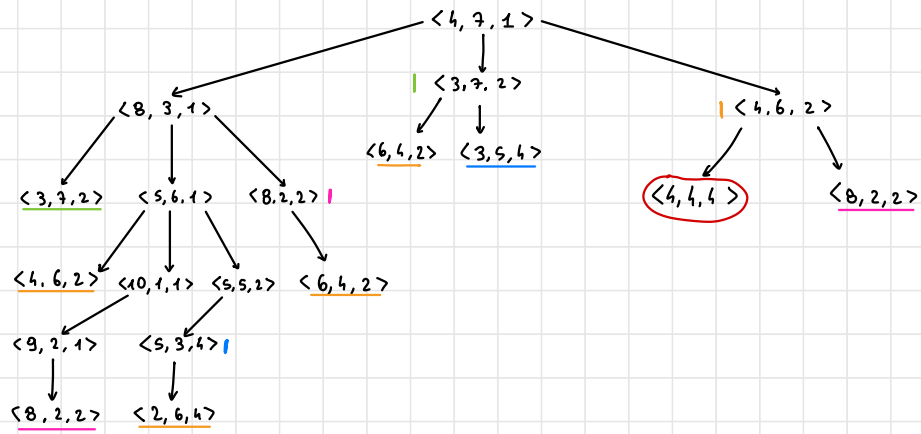
FINAL STATE : X - 1



- We have 12 coins in three piles. In the first pile we have 4 coins, in the second we have 7 of them, while the third we have a single coin.
- We have a rule for moving coins: we can move coins from a pile A to a pile B if coins in B double.

(ignoring loops and failures)

- Show the search space starting from the configuration <4,7,1> and the goal is <4,4,4>.



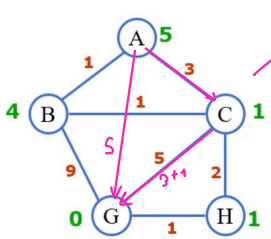
- Let us consider the following graph. Red numbers on arcs are costs while green numbers on nodes are heuristic values. Arcs are not oriended so they can be traversed in both directions. Consider A as the initial state and G as a goal.

Admissible ? Yes

Consistent ? No

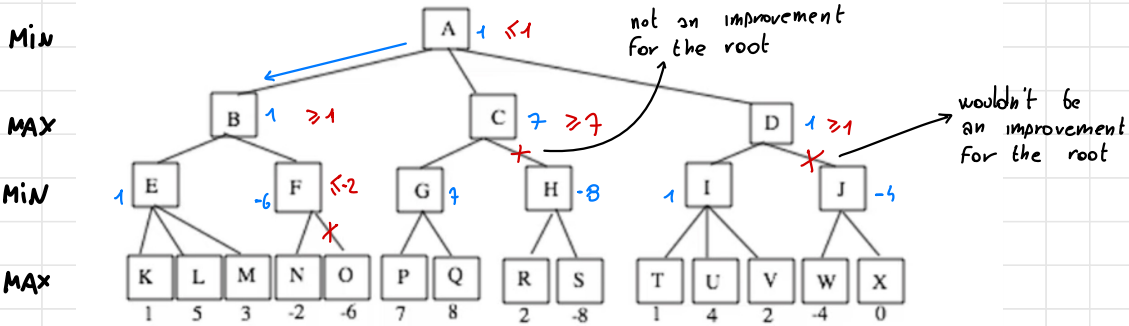
$5 \leq 3+1$ ×

- Answer to these questions:
  - Is the heuristic consistent and admissible?
  - Does the algorithm find the minimum cost path to the goal? If not what should I change in the heuristic to make it consistent?



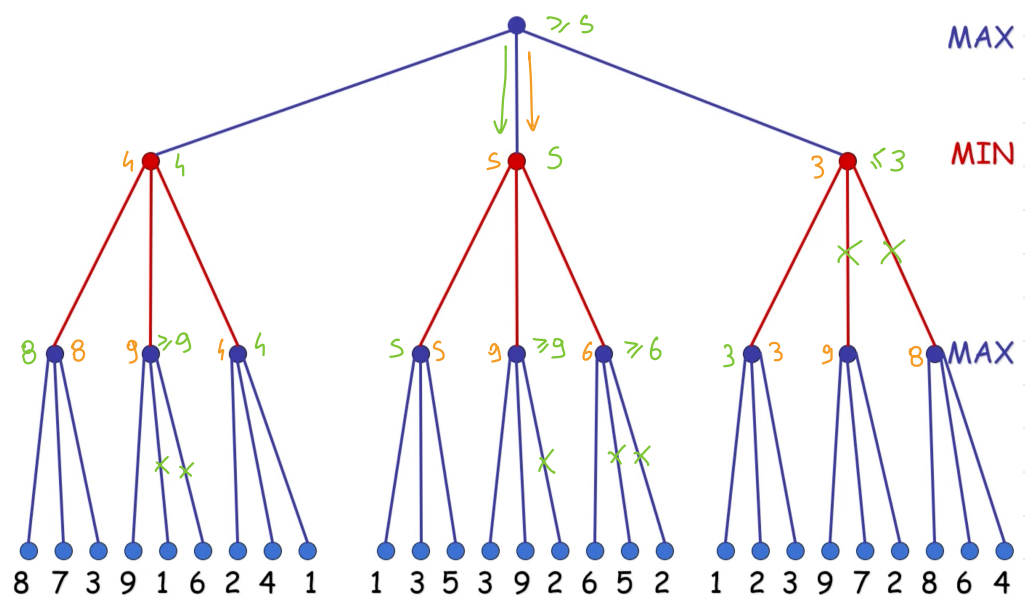
30/10/24

- Consider the following search tree representing a game. The evaluation of the leaves has been given by the first player.



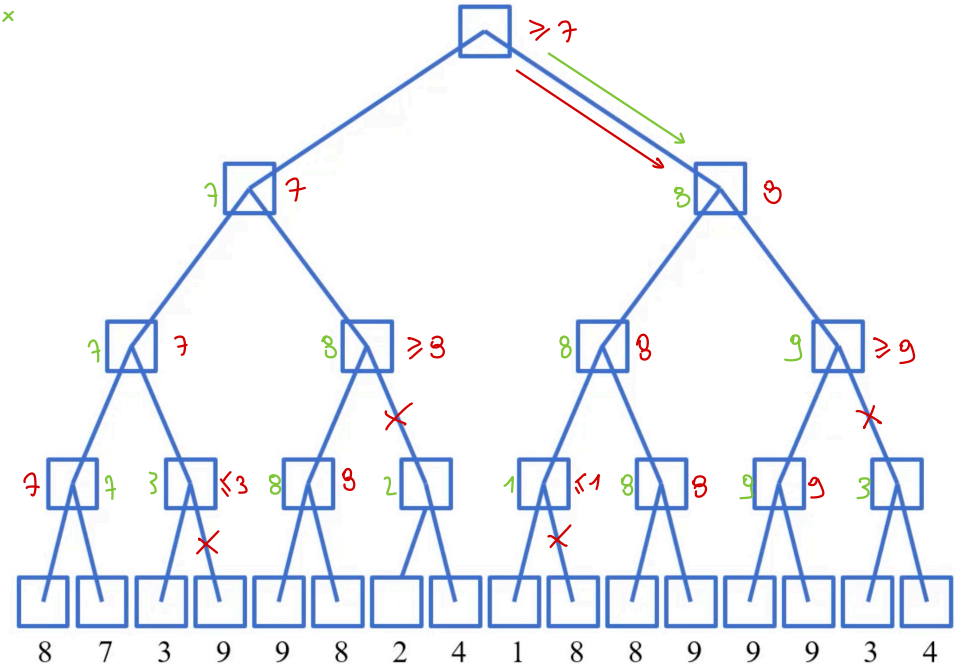
- Suppose that the first player is MIN, which move is the most convenient? First you have to apply the MIN-MAX algorithm. Then alfa beta cuts should be identified

04/11/24 ALPHA-BETA CUT MIN-MAX



ALPHA-BETA CUT

MIN-MAX



11 / 11 / 24

1) GREEN FORMULATION → KOWALSKI FORMULATION

stack(x,y)

PREC: holding(x), clear(y)

EFFECT:

ADD on(x,y), handempty, clear(x)

DELETE holding(x), clear(y)

EFFECTS

PRECONDITIONS

FRAME ACTIONS

```
holds( on(X,Y), do( stack(X,Y), S)).
holds( handempty, do( stack(X,Y), S)).
holds( clear(X), do( stack(X,Y), S)).

pact( stack(X,Y), S) :-
    holds( holding(X), S),
    holds( clear(Y), S).

holds(V, do(stack(X,Y),S)) :-
    holds(V,S),
    V \= holding(X),
    V \= clear(Y)
```

2) INITIAL STATE + ACTIONS → (by STRIPS) GOAL

INITIAL STATE

```
at(P1, rome)
at(P3, milan)
at(P2, bologna)
connected(rome, bologna)
connected(bologna, rome)
connected(milan, bologna)
connected(bologna, milan)
```

ACTIONS

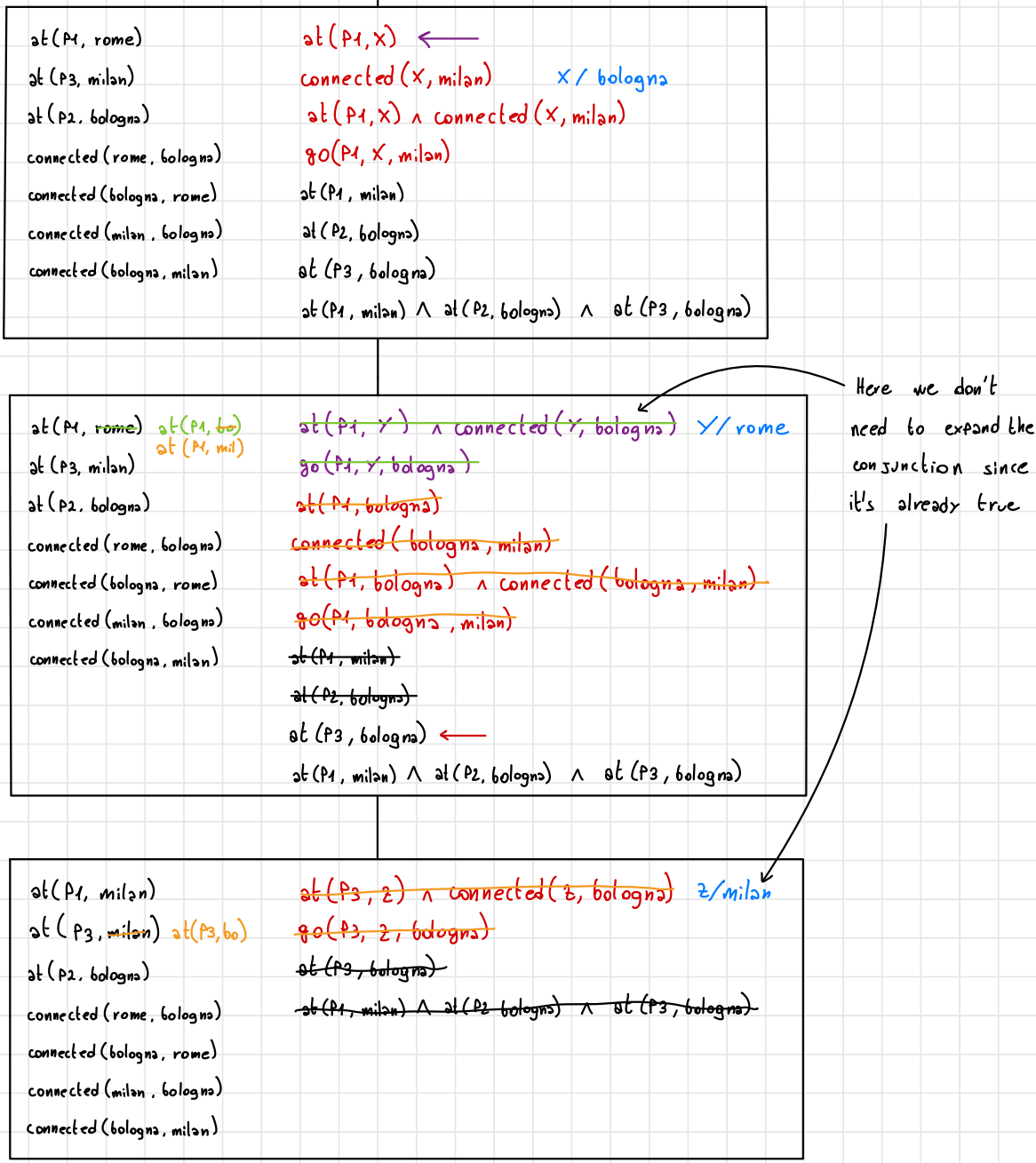
```
go(P, X, Y):
    PREC : at(P,X),
           connected(X,Y)
    DELETE : at(P,X)
    ADD :    at(P,Y)
```

GOAL

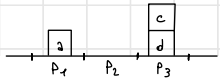
```
at(P1, milan)
at(P2, bologna)
at(P3, bologna)
```

INITIAL STATE

at(P1, rome)	at(P1, milan) ←
at(P3, milan)	at(P2, bologna)
at(P2, bologna)	at(P3, bologna)
connected(rome, bologna)	at(P1, milan) ∧ at(P2, bologna) ∧ at(P3, bologna)
connected(bologna, rome)	
connected(milan, bologna)	
connected(bologna, milan)	


at(P1, rome)
at(P3, milan)
at(P2, bologna)
connected(rome, bologna)
connected(bologna, rome)
connected(milan, bologna)
connected(bologna, milan)
at(P1, X) ←
connected(X, milan) X / bologna
at(P1, X) ∧ connected(X, milan)
go(P1, X, milan)
at(P1, milan)
at(P2, bologna)
at(P3, bologna)
at(P1, milan) ∧ at(P2, bologna) ∧ at(P3, bologna)
at(P1, rome) at(P1, bo) at(P1, mil)
at(P3, milan)
at(P2, bologna)
connected(rome, bologna)
connected(bologna, rome)
connected(milan, bologna)
connected(bologna, milan)
at(P1, Y) ∧ connected(Y, bologna) Y / rome
go(P1, Y, bologna)
at(P1, bologna)
connected(bologna, milan)
at(P1, bologna) ∧ connected(bologna, milan)
go(P1, bologna, milan)
at(P1, milan)
at(P2, bologna)
at(P3, bologna) ←
at(P1, milan) ∧ at(P2, bologna) ∧ at(P3, bologna)
Here we don't need to expand the conjunction since it's already true
at(P1, milan)
at(P3, milan) at(P3, bo)
at(P2, bologna)
connected(rome, bologna)
connected(bologna, rome)
connected(milan, bologna)
connected(bologna, milan)
at(P3, z) ∧ connected(z, bologna) z/milan
go(P3, z, bologna)
at(P3, bologna)
at(P1, milan) ∧ at(P2 bologna) ∧ at(P3, bologna)

13/11/24



Given the initial state:
[ontable(a,p1), ontable(d,p3), on(c,d), clear(a), clear(c), empty(p2), handempty]
(p1,p2,p3 are three positions on the table)

GOAL : ontable (a, P2)

Actions are modelled as:

- **pickup(X,Pos)**
  PRECOND: ontable(X,Pos), clear(X), handempty
  DELETE: ontable(X,Pos), clear(X), handempty
  ADD: holding(X), empty(Pos)
- **putdown(X,Pos)**
  PRECOND: holding(X), empty(Pos)
  DELETE: holding(X), empty(Pos)
  ADD: ontable(X,Pos), clear(X), handempty
- **stack(X,Y)**
  PRECOND: holding(X), clear(Y)
  DELETE: holding(X), clear(Y)
  ADD: handempty, on(X,Y), clear(X)
- **unstack(X,Y)**
  PRECOND: handempty, on(X,Y), clear(X)
  DELETE: handempty, on(X,Y), clear(X)
  ADD: holding(X), clear(Y)

STATE : ontable (a, P1), ontable (d, P3), on (c, d), clear (a), clear (c), empty (P2), handempty
GOAL : ontable (a, P2)

↓

STATE : ontable (a, P1), ontable (d, P3), on (c, d), clear (a), clear (c), empty (P2), handempty
GOAL : holding (a), empty (P2), holding (a) ∧ empty (P2), putdown (a, P2), ontable (a, P2)

↓

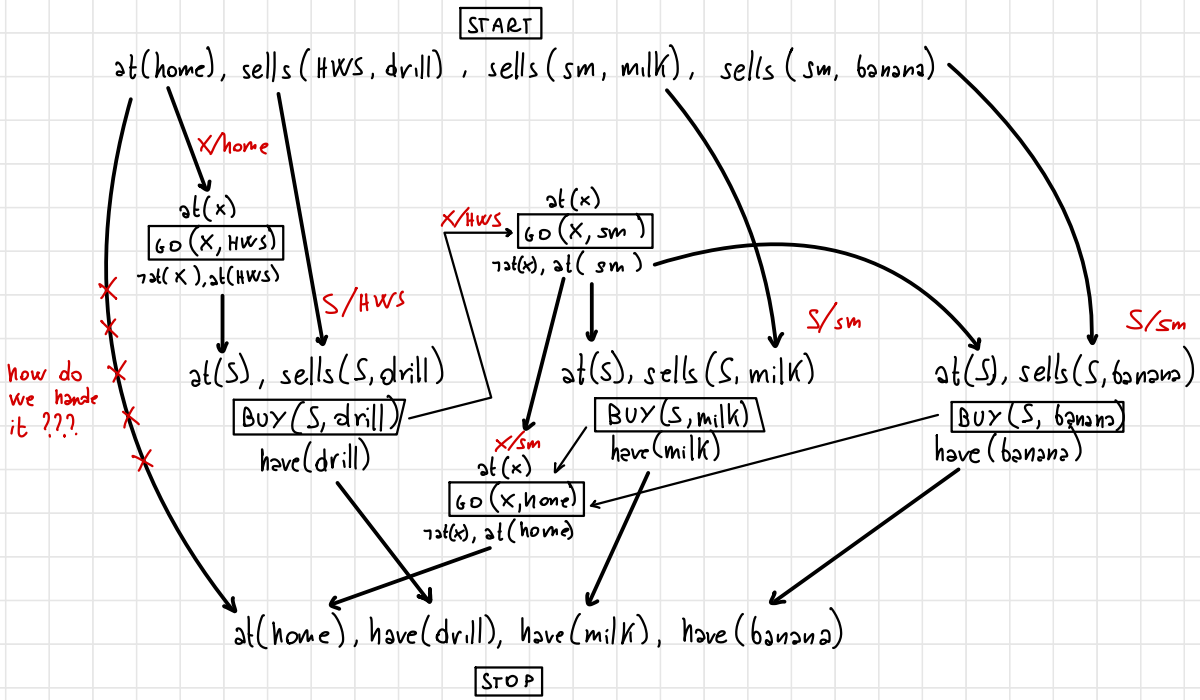
P/P1

STATE : ~~ontable (a, P1)~~, ontable (d, P3), on (c, d) ~~clear (a)~~, clear (c), ~~empty (P2)~~, ~~handempty~~, ~~holding (a)~~, empty (P1), ontable (a, P2), handempty, clear (a)
GOAL : ~~ontable (a, P) ∧ clear (a) ∧ handempty~~, pickup (a, P),
~~holding (a), empty (P2), holding (a) ∧ empty (P2)~~, ~~putdown (a, P2)~~,
ontable (a, P2)

↓

STATE : ontable (d, P3), on (c, d), clear (c), empty (P1), ontable (a, P2), handempty, clear (a)
GOAL : ~~ontable (a, P2)~~

- Initial state:
  at (home), sells (HWS, drill), sells (sm, milk), sells (sm, banana)
- Goal:
  at (home), have (drill), have (milk), have (banana)
- actions:
  Go (X, Y):
  PRECOND: at (X)
  EFFECT: at (Y), ¬ at (X)
  buy (S, Y):
  PRECOND: at (S), sells (S, Y)
  EFFECT: have (Y)



18/11/24

Given the following initial state **ontable(a,p1), ontable(d,p3), on(c,d), ontable (b,p2), clear(a), clear(c), clear(b), handempty**
and the goal **ontable(a,p2)**

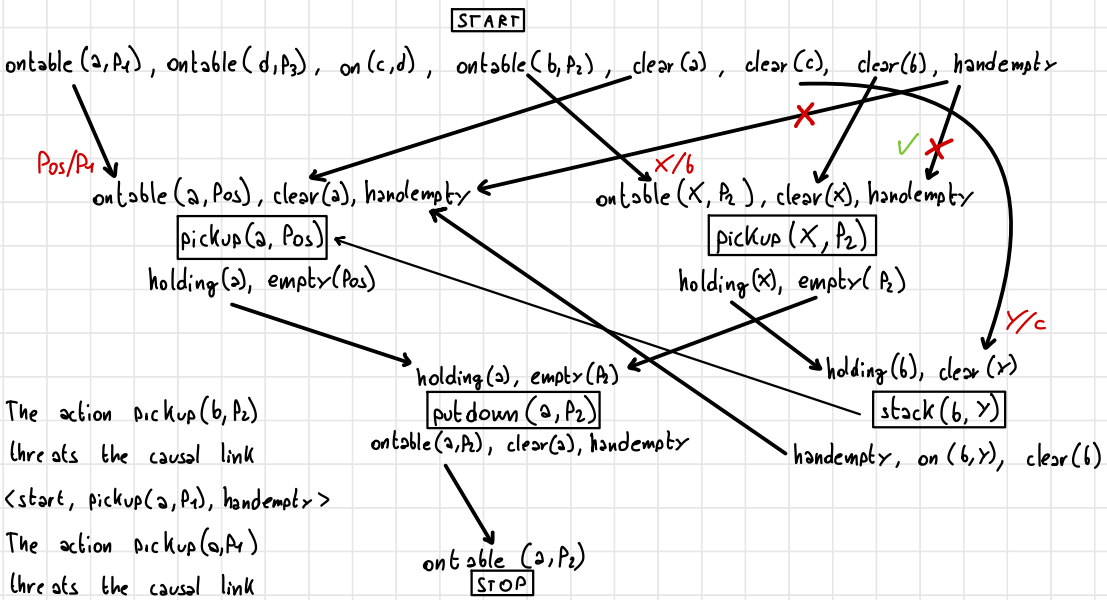
Actions are described as follows

pickup(X,Pos)
PRECOND: ontable(X,Pos), clear(X), handempty
DELETE: ontable(X,Pos), clear(X), handempty
ADD: holding(X), empty(Pos)

putdown(X,Pos)
PRECOND: holding(X), empty(Pos)
DELETE: holding(X), empty(Pos)
ADD: ontable(X,Pos), clear(X), handempty

stack(X,Y)
PRECOND: holding(X), clear(Y)
DELETE: holding(X), clear(Y)
ADD: handempty, on(X,Y), clear(X)

unstack(X,Y)
PRECOND: handempty, on(X,Y), clear(X)
DELETE: handempty, on(X,Y), clear(X)
ADD: holding(X), clear(Y)



✗ The action pickup(b,P2) threats the causal link <start, pickup(a,P1), handempty>

✗ The action pickup(a,P1) threats the causal link <start, pickup(b,P2), handempty>

✓ I order them by using the white-knight stack(b,c) and maintain the causal link <start, pickup(b,P2), handempty>

Given the following initial state **robot_at(loc_a), handempty, object_at(plate, loc_a), object_at(cube, loc_b),**

We have to reach the goal: **coloured(plate, blue), coloured(cube,red)**

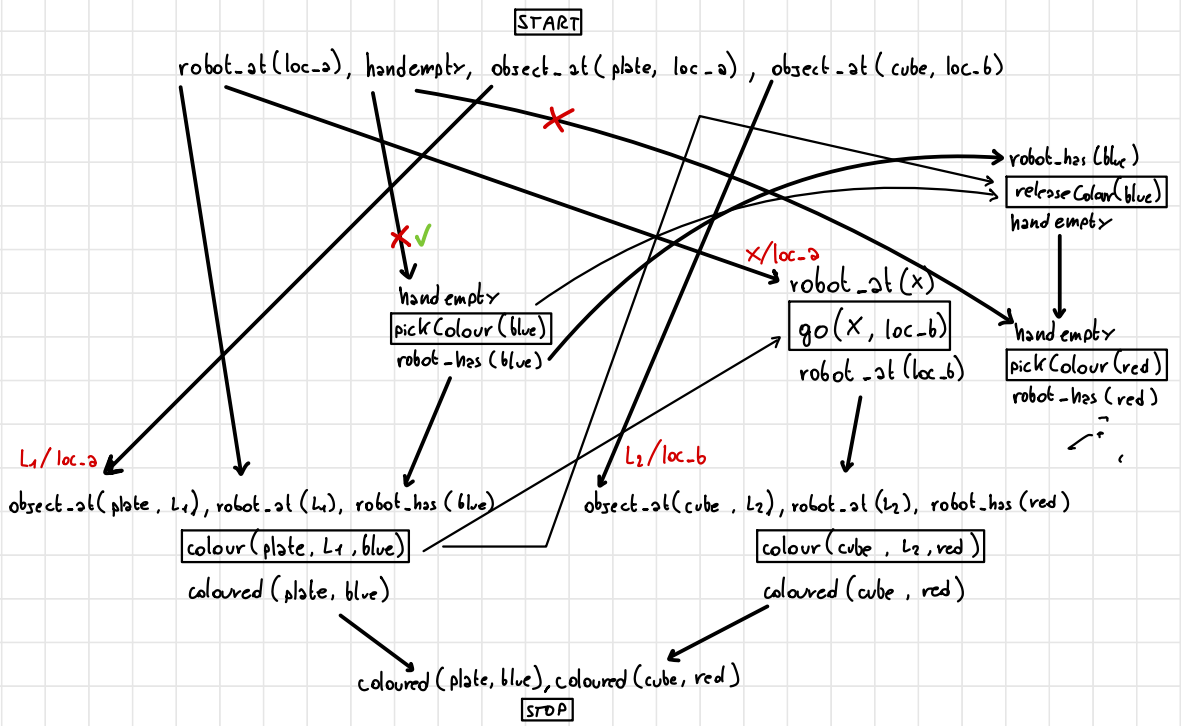
Actions are modelled as follows:

colour(Object, Location, Colour)
PRECOND: object_at(Object, Location), robot_at(Location), robot_has(Colour)
ADD: coloured(Object, Colour)

go(X,Y)
PRECOND: robot_at(X)
DELETE: robot_at(X)
ADD: robot_at(Y)

pickColour(C)
PRECOND: handempty
DELETE: handempty
ADD: robot_has(C)

releaseColour(C)
PRECOND: robot_has(C)
DELETE: robot_has(C)
ADD: handempty

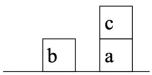


✗ pickColour(red) threats the causal link <start, pickColor(blue), handempty>

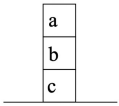
✗ pickColour(blue) threats the causal link <start, pickColor(red), handempty>

releaseColour(blue) threats the causal link <pickColour(blue), colour(plate, loc_a, blue), robot_has(b

Initial state ([*I*/*S*]):
clear (b), clear (c), on (c, a), ontable (a)
ontable (b), handempty.



Goal ([*G*]):
on (a,b), on (b, c).



NON SI CAPISCE LA CORREZIONE

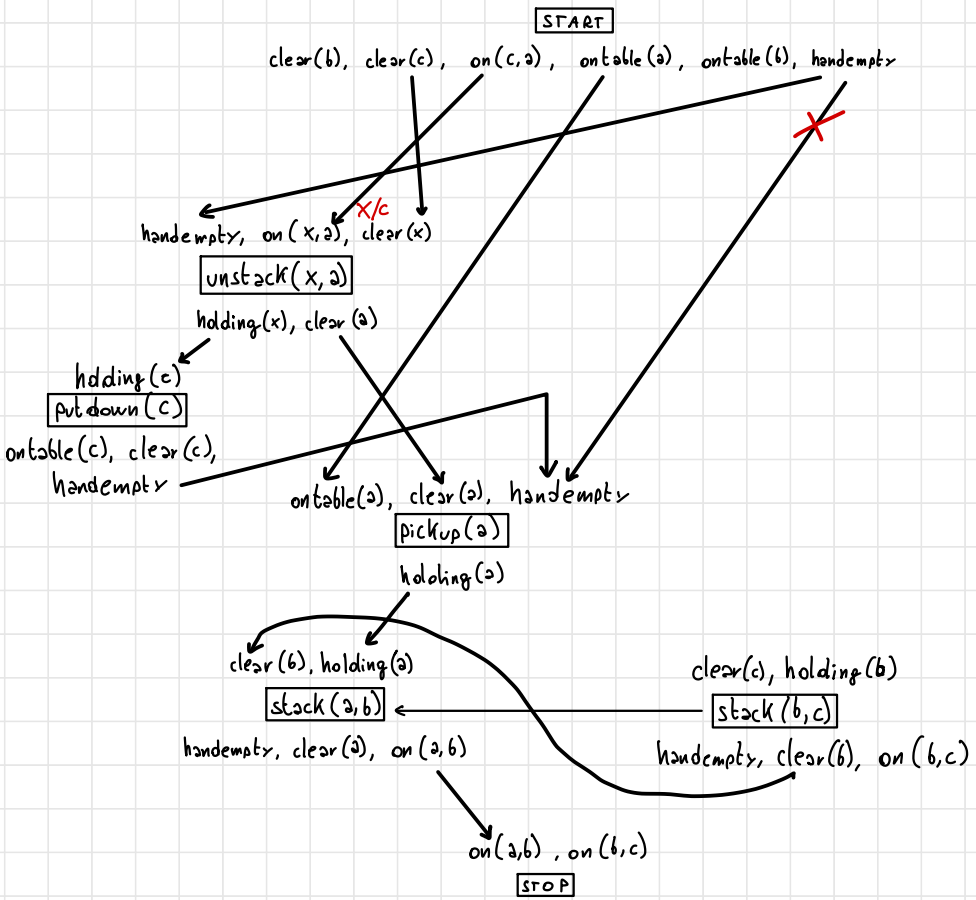
actions

pickup (X)
PRECOND: *ontable (X), clear (X), handempty*
POSTCOND: *holding (X) not ontable (X), not clear (X), not handempty*

putdown (X)
PRECOND: *holding (X)*
POSTCOND: *ontable (X), clear (X), handempty*

stack (X, Y)
PRECOND: *holding (X), clear (Y)*
POSTCOND: *not holding (X), not clear (Y), handempty, on (X, Y), clear (X)*

unstack (X, Y)
PRECOND: *handempty, on (X, Y), clear (X)*
POSTCOND: *holding (X), clear (Y) ,.. not handempty, not on (X, Y), not clear (X),*



Given the following initial state **[at(locationA), have_battery, handempty]**: and actions modelled as follows:

take_picture(Location)
PRECOND: have_battery, at(Location), have_camera
DELETE: have_battery
ADD: picture(Location)

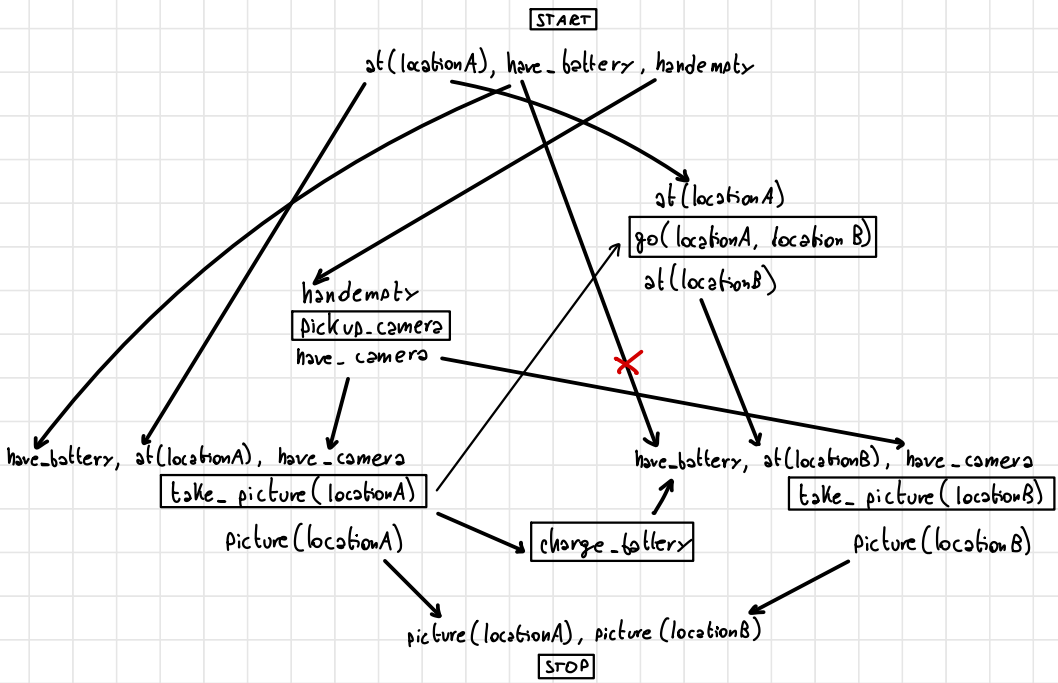
putdown_camera
PRECOND: have_camera
DELETE: have_camera
ADD: handempty

pickup_camera
PRECOND: handempty
DELETE: handempty
ADD: have_camera

charge_battery
PRECOND: not have_battery
DELETE: -
ADD have_battery

go(Location1, Location2)
PRECOND: at(Location1)
DELETE: at(Location1)
ADD at(Location2)

and the following goal **picture(locationA), picture(locationB)**



take_picture(locA) threats <start, take_picture(locB), have_battery>
take_picture(locB) threats <start, take_picture(locA), have_battery>
go(locA, locB) threats <start, take_picture(locA), at(locA)>

25/11/24

Given the following initial state **ontable(a,p1), ontable(d,p3), on(c,d), ontable (b,p2), clear(a), clear(c), clear(b), handempty**
and the goal **ontable(a,p2)**

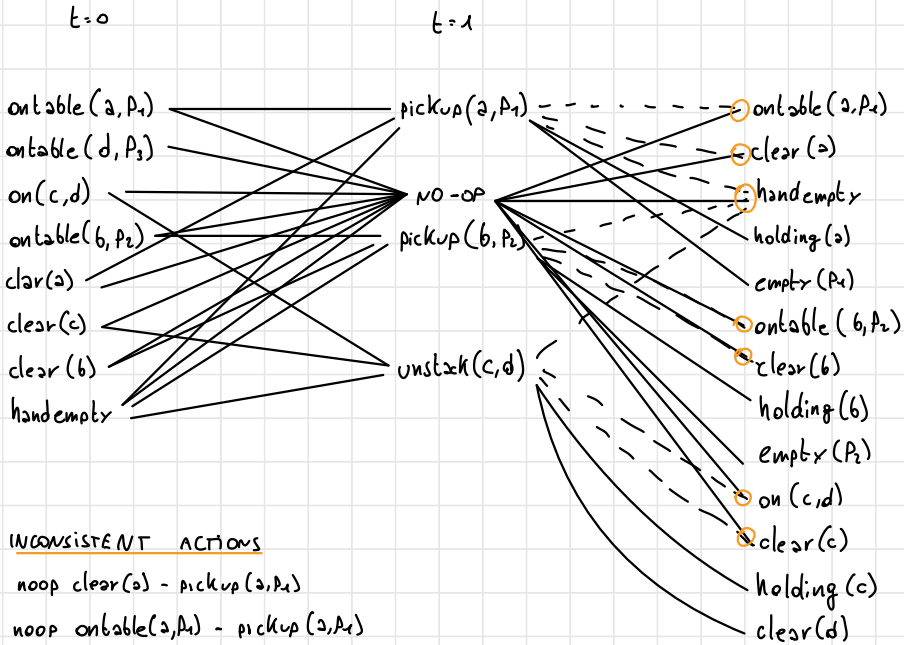
Actions are described as follows

pickup(X,Pos)
PRECOND: ontable(X,Pos), clear(X), handempty
DELETE: ontable(X,Pos), clear(X), handempty
ADD: holding(X), empty(Pos)

putdown(X,Pos)
PRECOND: holding(X), empty(Pos)
DELETE: holding(X), empty(Pos)
ADD: ontable(X,Pos), clear(X), handempty

stack(X,Y)
PRECOND: holding(X), clear(Y)
DELETE: holding(X), clear(Y)
ADD: handempty, on(X,Y), clear(X)

unstack(X,Y)
PRECOND: handempty, on(X,Y), clear(X)
DELETE: handempty, on(X,Y), clear(X)
ADD: holding(X), clear(Y)



INCONSISTENT ACTIONS

noop clear(a) - pickup(a,P1)

noop ontable(a,P1) - pickup(a,P1)

noop handempty - pickup(a,P1)

noop clear(b) - pickup(b,P2)

noop ontable(b,P2) - pickup(b,P2)

noop handempty - pickup(b,P2)

noop clear(b) - pickup(b,P2)

noop handempty - unstack(c,d)

noop on(c,d) - unstack(c,d)

noop clear(c) - unstack(c,d)

pickup(a,P1) - pickup(b,P2) - unstack(c,d) on handempty

Given the following initial state **robot_at(loc_a), handempty, object_at(plate, loc_a), object_at(cube, loc_b),**

We have to reach the goal: **coloured(plate, blue), coloured(cube,red)**

Actions are modelled as follows:

colour(Object, Location, Colour)
PRECOND: object_at(Object, Location), robot_at(Location), robot_has(Colour)
ADD: coloured(Object, Colour)

go(X,Y)
PRECOND: robot_at(X)
DELETE: robot_at(X)
ADD: robot_at(Y)

pickColour(C)
PRECOND: handempty
DELETE: handempty
ADD: robot_has(C)

releaseColour(C)
PRECOND: robot_has(C)
DELETE: robot_has(C)
ADD: handempty

2/12/24

Domain : $X_1, X_2, X_3 \in [1,2,3]$

Constraint: $X_1 < X_2 < X_3$

PARTIAL LOOK AHEAD

In X_1 we cutoff 3 since its the only one without support. ← no value in X_2 for $X_1 = 3$ such that $X_1 < X_2$

$X_1 \in [1, 2, ~~3~~]$

It's done the same with X_2.

$X_2 \in [1, 2, ~~3~~]$

FULL LOOK AHEAD

In X_1 we remove 3 since doesn't have support in X_2 for $X_1 < X_2$

In X_2 we remove 1 since doesn't have support in X_1 for $X_1 < X_2$

$X_1 \in [1, 2, ~~3~~]$

$X_2 \in [~~1~~, 2, 3]$

In X_2 we remove 3 since doesn't have support in X_2 for $X_2 < X_3$

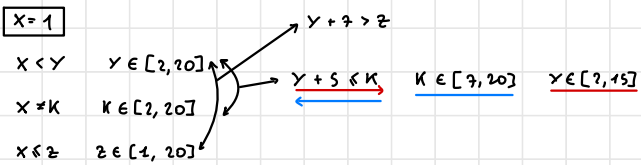
In X_3 we remove 1, 2 since doesn't have support in X_2 for $X_2 < X_3$

$X_2 \in [~~1~~, 2, ~~3~~]$

$X_3 \in [~~1~~, ~~2~~, 3]$

- Suppose we have the following constraints:
  $X < Y$, $X \neq K$, $Y + 5 \leq K$, $Y + 7 > Z$, $X \leq Z$
  defined on the variables X, Y, Z, K whose domain of definition is [1..20].

- Solve the problem by applying the strategy of full look ahead.



Y=2

1 < 2

2 + 5 ≤ K

2 + 7 > Z Z ∈ [1 , 8]

1 ≤ Z

Z=1 K ∈ [7, 20] doesn't change since there are no constraints between K and Z

K=7

We have 4 teachers a, b, c and d that must give 10 lessons. Each lesson can be held by a single teacher. For each lesson we know the starting time and duration Di = 2 hours. Also, we know that no teacher can hold two consecutive classes and even lessons overlapping temporally. Model the problem as a constraint satisfaction problem by defining the variables, the domains of the variables and constraints between these variables. Also assume that the lessons have the following starting times:

- S1 = 7
- S2 = 8
- S3 = 9
- S4 = 10
- S5 = 11
- S6 = 12
- S7 = 13
- S8 = 14
- S9 = 15
- S10 = 16

Model the problem as a CSP and solve it using the forward checking.

$L_1, \dots, L_{10} \in [a, b, c, d]$

$\forall i,j \in [1,10] : (S_i + 2 = S_j) \longrightarrow (L_i \neq L_j)$

$\forall i,j \in [1,10] : (S_i + 2 > S_j) \wedge (S_i + 2 \leq S_j + 2) \longrightarrow (L_i \neq L_j)$

$X_1 = a$

$X_2 = [b, c, d]$, $X_3 = [b, c, d]$

$X_2 = b$

$X_3 = [c, d]$, $X_4 = [a, c, d]$

$X_3 = c$

$X_4 = [a, d]$, $X_5 = [a, b, d]$

$X_4 = a$

$X_5 = [b, d]$, $X_6 = [b, c, d]$

$X_5 = b$

$X_6 = [c, d]$, $X_7 = [a, c, d]$

$X_6 = c$

$X_7 = [a, d]$, $X_8 = [a, b, d]$

$X_7 = a$

$X_8 = [b, d]$, $X_9 = [b, c, d]$

$X_8 = b$

$X_9 = [c, d]$, $X_{10} = [a, c, d]$

$X_9 = c$

$X_{10} = [a, d]$

$X_{10} = a$

9/12/24

Given the following constraints:

```
A :: [1..4], B :: [1..4], C :: [2,4,6] D :: [1,7,9], E :: [2..5 ]
A> D, A ≠ B, B < C, C > D, E = A
```

Draw the graph corresponding to the constraint satisfaction problem and apply arc-consistency. Show the search tree to get to the first solution using as a heuristic of the first-fail assignment of values to variables (MRV) is at each instantiation apply the arc-consistency to the remaining network.

A :: [1..4]
B :: [1..4]
C :: [2,4,6]
D :: [1,7,9]
E :: [2..5]

A > D ⟶ A :: [2,3,4] , D :: [1]
A ≠ B
D < C
C > D
E = A ⟶ E :: [2,3,4]

First fail:

D = 1

A = 2

E = 2

B = 1

C = 2

A: [1..6] B: [1..6] C: [1..6]

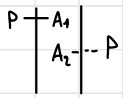
A = 1 Fail on A ≥ B+1

A = 2 B = [1] C = [1..6]

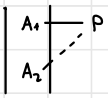
A = 2 B = 1 C = [1..4]

A = 2 B = 1 C = 1

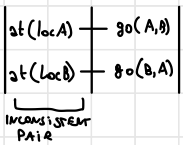
INTERFERENCE (A_1 - A_2)

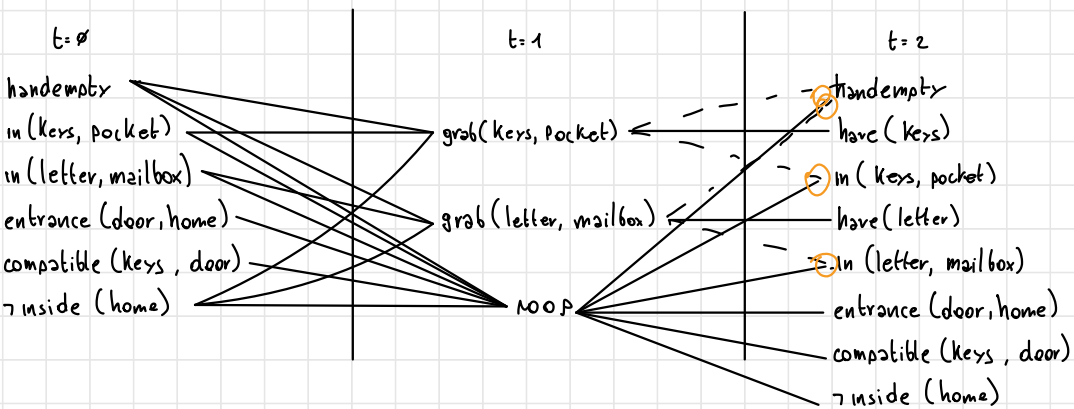


INCONSIST EFFECTS (A_1 - A_2)



COMPETING NEEDS





INCOMPATIBLE ACTIONS

NOOP handempty	and	grab(keys, pocket)	inconsistent effects
NOOP handempty	and	grab(letter, mailbox)	
NOOP in(keys, pocket)	and	grab(keys, pocket)	
NOOP in(letter, mailbox)	and	grab(letter, mailbox)	
grab(keys, pocket)	and	grab(letter, mailbox)	interference

INCOMPATIBLE PROPOSITIONS

have(keys) - handempty
have(keys) - in(keys, pocket)
have(letter) - handempty
have(letter) - in(letter, mailbox)
have(letter) - have(keys)

SUSSMAN ANOMALY on STRIPS

INITIAL STATE

clear (b)

clear (c)

on(c, a)

ontable(a)

ontable(b)

handempty

GOAL STATE

on (a,b)

on (c,d)

on(a,b) ∧ on(b,c)

no, so decompose

are ther true ?

INITIAL STATE

*

GOAL STATE

holding(a)

~~clear (b)~~ TRUE IN THE STATE

holding (a) ∧ clear(b)

STACK(a,b)

on (c,d)

on(a,b) ∧ on(b,c)

INITIAL STATE

*

GOAL STATE

pickup(a)

holding(a)

holding (a) ∧ clear(b)

STACK(a,b)

on (c,d)

on(a,b) ∧ on(b,c)

INITIAL STATE

*

GOAL STATE

~~ontable (a)~~

clear (a)

~~handempty~~

ontable (a) ∧ clear (a) ∧ handempty

no, so decompose

are ther true ?

pickup(a)

holding(a)

holding (a) ∧ clear(b)

STACK(a,b)

on (c,d)

on(a,b) ∧ on(b,c)

INITIAL STATE

GOAL STATE

~~on(X, a) ∧ clear(X) ∧ handempty~~ TRUE IF X/c

UNSTACK(X, a)

clear(a)

ontable(a) ∧ clear(a) ∧ handempty

pickup(a)

holding(a)

holding(a) ∧ clear(b)

STACK(a,b)

on(c,d)

on(a,b) ∧ on(b,c)

INITIAL STATE

~~clear(b)~~

clear(c)

~~on(c,a)~~

ontable(a)

ontable(b)

~~handempty~~

clear(a)

holding(c)

GOAL STATE

~~UNSTACK(C, a)~~ apply unstack

~~clear(a)~~

ontable(a) ∧ clear(a) ∧ handempty

pickup(a)

holding(a)

holding(a) ∧ clear(b)

STACK(a,b)

on(c,d)

on(a,b) ∧ on(b,c)

INITIAL STATE

clear(c)

ontable(a)

ontable(b)

clear(a)

holding(c)

GOAL STATE

handempty ← add since it's not true anymore

ontable(a) ∧ clear(a) ∧ handempty

pickup(a)

holding(a)

holding(a) ∧ clear(b)

STACK(a,b)

on(c,d)

on(a,b) ∧ on(b,c)

INITIAL STATE	GOAL STATE
clear(c)	~~holding(X)~~ X/c
ontable(a)	PUTDOWN(X)
ontable(b)	handempty
clear(a)	ontable(a) ∧ clear(a) ∧ handempty
holding(c)	PICKUP(a)
	holding(a)
	holding(a) ∧ clear(b)
	STACK(a,b)
	on(c,d)
	on(a,b) ∧ on(b,c)

INITIAL STATE	GOAL STATE
clear(c)	~~PUTDOWN(c)~~
ontable(a)	handempty
ontable(b)	ontable(a) ∧ clear(a) ∧ handempty
clear(a)	PICKUP(a)
~~holding(c)~~	holding(a)
ontable(c)	holding(a) ∧ clear(b)
	STACK(a,b)
	on(c,d)
	on(a,b) ∧ on(b,c)

ES.2

5	6
3	4
1	2

A, B, C, D, E, F :: [1..3]

A ≠ C

B > E

F > E

D > C

A = E

- ARC CONSISTENCY

1st ROUND

A ≠ C

B > E [~~1~~, 2, 3] (B) [1, 2, ~~3~~] (E)

F > E [~~1~~, 2, 3] (F) [1, 2] (E)

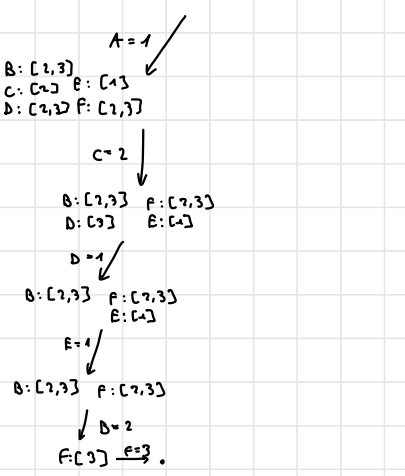
D > C [1, 2, 3] [1, 2, ~~3~~] (C)

A = E [1, 2, ~~3~~] (A) [1, 2]

2nd ROUND

A ≠ C	✓	A : [1,2]
B > E	✓	B : [2,3]
F > E	✓	C : [1,2]
D > C	✓	D : [2,3]
A = E	✓	E : [1,2]
		F :: [1,3]

- FORWARD CHECKING



ESERCIZIO IMPLICATO

$A :: [1..10]$	$A > B$
$B :: [4..15]$	$A \neq C$
$C :: [3..21]$	$C > B$
$D :: [1,2]$	$D + 14 = B$

- ARC CONSISTENCY